

NOISE AND FILTERS

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from matplotlib.image import imread
import cv2
import seaborn as sns

image = cv2.imread('conor.jpg')
image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

plt.figure(figsize=(15, 9))

def add_noise(image, method, idx, mean=0, std=25, low=50, high=100,
scale=20, shape=2, noise_ratio=0.05):
    noisy_image = image.copy()

    if method == "gaussian":
        noise = np.random.normal(mean, std, image.shape)
        noisy_image = image + noise
    elif method == "rayleigh":
        noise = np.random.rayleigh(scale, image.shape)
        noisy_image = image + noise
    elif method == "gamma":
        noise = np.random.gamma(shape, scale, image.shape)
        noisy_image = image + noise
    elif method == "exponential":
        noise = np.random.exponential(scale, image.shape)
        noisy_image = image + noise
    elif method == "uniform":
        noise = np.random.uniform(low, high, image.shape)
        noisy_image = image + noise
    elif method == "salt and pepper":
        noisy_image = impulse(image, noise_ratio)
    else:
        print("Invalid Noise method")
        return image

    noisy_image = np.clip(noisy_image, 0, 255).astype(np.uint8)

    plt.subplot(2, 3, idx)
    plt.imshow(noisy_image, cmap='gray')
    plt.title(f"{method.capitalize()} Noise")
    plt.axis('off')

    return noisy_image
```

```
def impulse(image, noise_ratio):
    noisy_image = image.copy()
    h, w = noisy_image.shape
    noisy_pixels = int(h * w * noise_ratio)

    for _ in range(noisy_pixels):
        row, col = np.random.randint(0, h), np.random.randint(0, w)
        noisy_image[row, col] = 0 if np.random.rand() < 0.5 else 255

    return noisy_image

n_img = add_noise(image, 'gaussian', idx=1)
u_img = add_noise(image, 'uniform', idx=2)
r_img = add_noise(image, 'rayleigh', idx=3)
g_img = add_noise(image, 'gamma', idx=4)
e_img = add_noise(image, 'exponential', idx=5)
i_img = add_noise(image, 'salt and pepper', idx=6)

plt.tight_layout()
plt.show()
```

Gaussian Noise



Uniform Noise



Rayleigh Noise



Gamma Noise



Exponential Noise



Salt and pepper Noise



```
from scipy.ndimage import generic_filter

def arithmetic_mean_filter(img, size=3):
    kernel = np.ones((size, size), dtype=np.float32) / (size * size)
    return cv2.filter2D(img, -1, kernel)

# Geometric Mean Filter
def geometric_mean_filter(img, size=3):
    def geo_mean(window):
```

```

        window = window[window > 0] # Avoid log(0)
        return np.exp(np.mean(np.log(window))) if len(window) > 0 else
1 # Prevent NaN
        return generic_filter(img.astype(np.float32), geo_mean,
size=(size, size)).astype(np.uint8)

# Harmonic Mean Filter
def harmonic_mean_filter(img, size=3):
    def harm_mean(window):
        window = window[window > 0] # Avoid division by zero
        return len(window) / np.sum(1.0 / window) if np.any(window >
0) else 1 # Prevent NaN
    return generic_filter(img.astype(np.float32), harm_mean,
size=(size, size)).astype(np.uint8)

# Median Filter
def median_filter(img, size=3):
    return cv2.medianBlur(img, size)

# Min Filter
def min_filter(img, size=3):
    return cv2.erode(img, np.ones((size, size), np.uint8))

# Max Filter
def max_filter(img, size=3):
    return cv2.dilate(img, np.ones((size, size), np.uint8))

# Midpoint Filter
def midpoint_filter(img, size=3):
    max_filt = max_filter(img, size)
    min_filt = min_filter(img, size)
    return ((max_filt + min_filt) / 2).astype(np.uint8)

# Load and Preprocess Image
img = cv2.imread("conor.jpg")
img = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY )
if img is None:
    raise ValueError("Error: Image not found or unable to load. Check
the file path!")

# Apply Rayleigh Noise
noisy_img = r_img

# Apply Restoration Filters
arith_mean = arithmetic_mean_filter(noisy_img)
geo_mean = geometric_mean_filter(noisy_img)
harm_mean = harmonic_mean_filter(noisy_img)
median_filt = median_filter(noisy_img)
min_filt = min_filter(noisy_img)
max_filt = max_filter(noisy_img)

```

```

midpoint_filt = midpoint_filter(noisy_img)

# Titles for Display
img_title = ['Original', 'Rayleigh Noise', 'Arithmetic Mean',
             'Geometric Mean', 'Harmonic Mean', 'Median', 'Min', 'Max', 'Midpoint']

# Images List
images = [img, noisy_img, arith_mean, geo_mean, harm_mean,
          median_filt, min_filt, max_filt, midpoint_filt]

# Plot Images
fig, axes = plt.subplots(3, 3, figsize=(12, 10))
for ax, im, title in zip(axes.flat, images, img_title):
    ax.imshow(im, cmap='gray')
    ax.set_title(title)
    ax.axis("off")

plt.tight_layout()
plt.show()

```

Original



Rayleigh Noise



Arithmetic Mean



Geometric Mean



Harmonic Mean



Median



Min



Max



Midpoint



```

from skimage.metrics import peak_signal_noise_ratio as psnr,
structural_similarity as ssim
import pandas as pd

# === Function to compute pixel-level differences ===
def compare_images(original, filtered, title):
    diff = cv2.absdiff(original, filtered)
    mae = np.mean(diff)
    psnr_val = psnr(original, filtered)
    ssim_val = ssim(original, filtered)
    return {
        'Filter': title,
        'MAE': round(mae, 2),
        'PSNR': round(psnr_val, 2),
        'SSIM': round(ssim_val, 4)
    }

# === Document changes for all filters ===
comparison_metrics = []
filters_to_compare = zip(images[2:], img_title[2:]) # Skip Original &
Noisy

for filtered_img, title in filters_to_compare:
    result = compare_images(img, filtered_img, title)
    comparison_metrics.append(result)

# === Convert to DataFrame ===
df_metrics = pd.DataFrame(comparison_metrics)

print("□ Filter Comparison (against original):\n")
print(df_metrics)

# === Optional: Visualize Difference Images ===
fig, axes = plt.subplots(2, 4, figsize=(16, 8))
for ax, (filtered_img, title) in zip(axes.flat, zip(images[2:],
img_title[2:])):
    diff_img = cv2.absdiff(img, filtered_img)
    ax.imshow(diff_img, cmap='hot')
    ax.set_title(f'Diff: {title}')
    ax.axis("off")

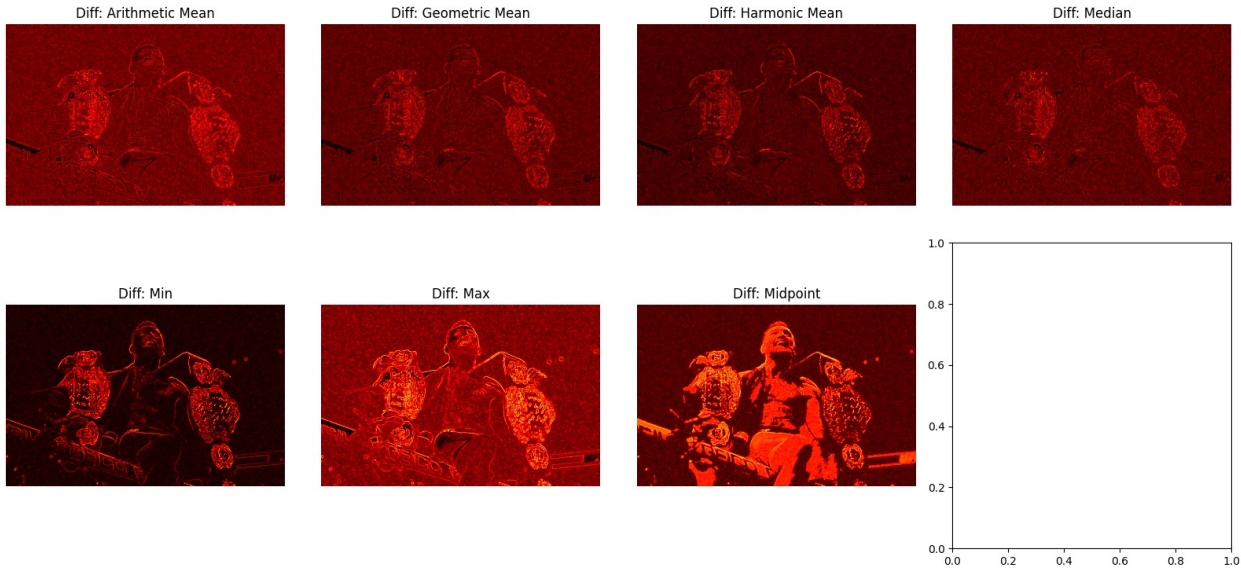
plt.tight_layout()
plt.show()

```

□ Filter Comparison (against original):

	Filter	MAE	PSNR	SSIM
0	Arithmetic Mean	26.19	18.94	0.5768
1	Geometric Mean	23.32	19.87	0.5974
2	Harmonic Mean	21.57	20.17	0.6013

3	Median	24.72	19.40	0.5677
4	Min	14.96	19.00	0.6239
5	Max	55.54	12.09	0.3254
6	Midpoint	46.48	12.54	0.2532



```
# === Histogram Plotting for Comparison ===
def plot_histograms(images, titles):
    plt.figure(figsize=(15, 8))
    for i, (img, title) in enumerate(zip(images, titles)):
        plt.subplot(3, 3, i+1)
        plt.hist(img.ravel(), bins=256, range=[0, 256], color='gray')
        plt.title(f'Histogram - {title}')
        plt.xlabel('Pixel Intensity')
        plt.ylabel('Frequency')
        plt.tight_layout()
    plt.show()

plot_histograms(images, img_title)
```

